

LAB 5: PARALLEL PROGRAMMING

I. Objectives

- Students understand the concept of parallel programming, differences between parallel programming model and steps to design parallel program.
- Students can apply knowledges to design and program required tasks on parallel programming.

II. Basic theory

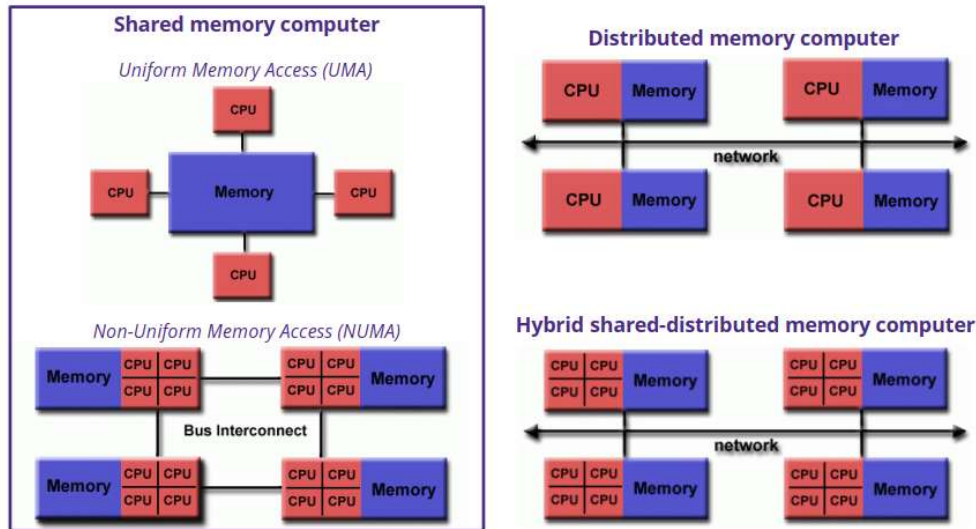
1. Introduction to parallel programming

Parallel programming is a way of solving a problem by dividing it into multiple parts that can run simultaneously on different processors. This approach contrasts with serial computing, where a problem is tackled one instruction at a time on one processor. Parallel programming provides key benefits:

- Improved performance:
- Better resource use:
- Handles complex tasks

A parallel computer is a system that uses multiple processors (CPUs) that communicate and cooperate solve a problem faster than a traditional single-processor machine. Memory architecture of a parallel computer determines how data is shared or distributed between processors — which impacts speed, efficiency, and programming complexity and can be divided in three types:

- Shared memory computer: All CPUs share a single, global memory space. Each CPU can read/write to the same memory, and changes made by one CPU are immediately visible to others. It contains two subtypes: Uniform memory access (All CPUs access memory at the same speed) and non-uniform memory access (Each CPU has its own local memory (faster access), but can still access other CPUs' memory (slower))
- Distributed memory computer: Each CPU has its own private memory and CPUs don't directly access each other's memory —they communicate with each other via network links (e.g., MPI - Message Passing Interface) to share data.
- Hybrid shared-distributed memory computer: Combines shared memory within small groups of CPUs and distributed memory between those groups — combining the best of both worlds.



Memory architecture of a parallel computer

2. Parallel programming models

A parallel programming model provides a framework for writing programs that can run in parallel guiding how processors share data, synchronize, and communicate. Below are several main types of models.

Thread Programming Model

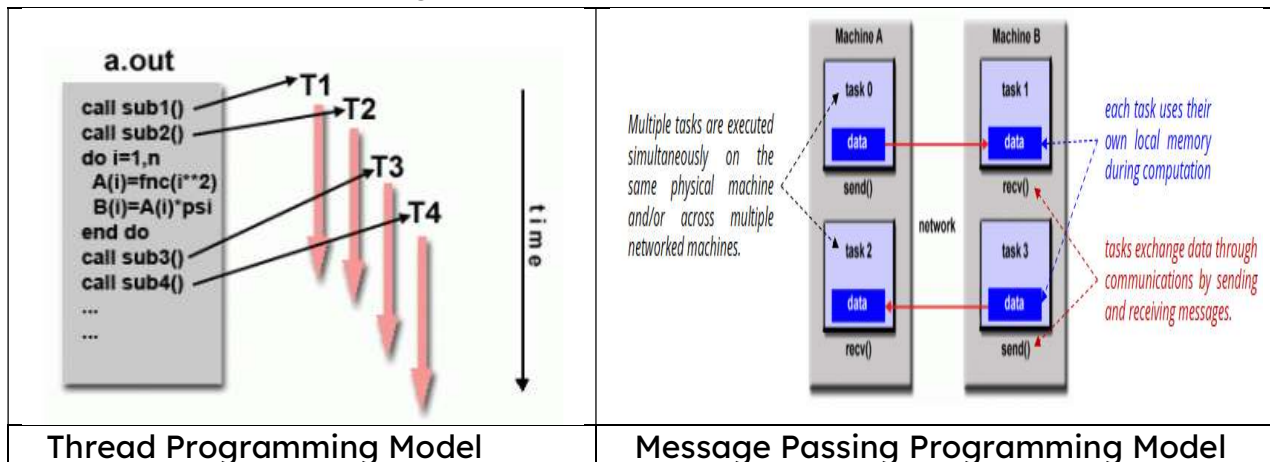
In thread programming model, a single process spawns multiple threads — each thread runs independently on different parts of the task but shares the same memory space (global memory). This model has fast communication between threads (shared memory, no message passing), efficiency (threads are lighter than processes) and be good for multi-core CPUs (threads run in parallel without duplicating memory). Besides, thread programming model face these challenges:

- Load balancing:
- Synchronization and performance overhead: If multiple threads access shared data at the same time, conflicts occur — needing locks or barriers (synchronization). However, Synchronization slows performance, especially when many threads wait on each other.
- Scalability: The model works well for a few threads, but as you add more, memory contention increases (too many threads competing for shared memory). For large-scale tasks, distributed memory models (e.g., message passing) might perform better

Message Passing Programming Model

This model is designed for distributed memory systems, where each processor has its own local memory. The program is split to multiple tasks, working in parallel across multiple processors. Tasks exchange data by sending and receiving messages over a network (even if it's within the same machine). Message passing programming model has high scalability (can run across thousands of machines); no race conditions (each task has its own memory, so no need for complex synchronization like in shared memory models) and fault-tolerant (If one machine fails, others keep running). Besides, this model face these challenges:

- Communication complexity & performance overhead: Sending/receiving data between machines adds latency and bandwidth costs. If too many tasks communicate at once, the system slows down.
- Scalability: The model scales well if communication is minimized. But Too much data exchange causes network bottlenecks.



Hybrid Programming Models

A hybrid programming model combines two or more parallel programming models to take advantage of their strengths and overcome limitations. It's designed for high-performance computing (HPC) environments — like supercomputers or large data clusters — to handle complex, large-scale problems efficiently. For example, hybrid programming model combines between thread and message passing programming model. They handle shared memory within a single machine (node) using multithreading. Different machines communicate by sending and receiving messages.

3. Parallel program design

To design parallel program, we can use Foster's methodology, a widely used strategy to break down the design of parallel programs into four key steps:

- Partitioning: this step breaks the computation into small, manageable tasks with the aim for many independent tasks to maximize parallelism. Each task handles a piece of data or a specific computation.
- Communication: Determine how tasks will exchange data — because many problems depend on shared results. This could involve sending and receiving messages (e.g., MPI) or shared memory (e.g., OpenMP). Because too much communication causes overhead — the goal is minimizing unnecessary communication.
- Agglomeration: Combine tasks where feasible to reduce overhead and improve efficiency. This may trade off some parallelism for better performance.
- Mapping: Assign tasks to threads or processes to balance the workload across hardware resources. There are two types of mapping: static mapping (Tasks are pre-assigned before execution, good when workloads are balanced and predictable) and dynamic mapping (tasks are assigned during execution, good for unpredictable tasks when workloads vary).

For an example, summing number from 0, ..., n-1 in parallel. We will perform 4 steps in Foster method to solve this problem.

Step 1: Partitioning

We will Split the array summation problem into independent tasks by dividing the array into equal chunks, one for each thread. Each thread sums a different range of numbers: thread 0 sums [0 to $n/4 - 1$], thread 1 sums [$n/4$ to $n/2 - 1$], and so on.

Step 2: Communication

Each thread calculate local sum and add local sum to global sum protected by mutex

Step 3: Agglomeration

In this example, each thread sums multiple numbers in its chunk instead of handling one number per thread (which would cause massive overhead).

Step 4: Mapping

This example using static mapping because equal workload per thread

Example:

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#define NUM_THREADS 4
#define RANGE 10000000
```

```

long long total_sum = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *sum_range(void *arg) {
    int *range = (int *)arg;
    int start = range[0];
    int end = range[1];
    long long local_sum = 0;

    for (int i = start; i <= end; i++) {
        local_sum += i;
    }

    pthread_mutex_lock(&lock);
    total_sum += local_sum;
    pthread_mutex_unlock(&lock);

    free(arg);
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int chunk = RANGE / NUM_THREADS;

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        int *range = malloc(2 * sizeof(int));
        range[0] = i * chunk + 1;
        if ((i == NUM_THREADS - 1))
            range[1] = RANGE;
        else
            range[1] = (i+1) * chunk;
        pthread_create(&threads[i], NULL, sum_range, range);
    }

    // Wait for all threads to finish
    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);

    printf("Total sum from 1 to %d: %lld\n", RANGE, total_sum);
}

```

```
    return 0;  
}
```